

Computer Networks Project — Phase 1

Private LAN Infrastructure: DNS, Load Balancing, HTTPS/TLS, Caching & Packet Analysis

Team Jarvis 07

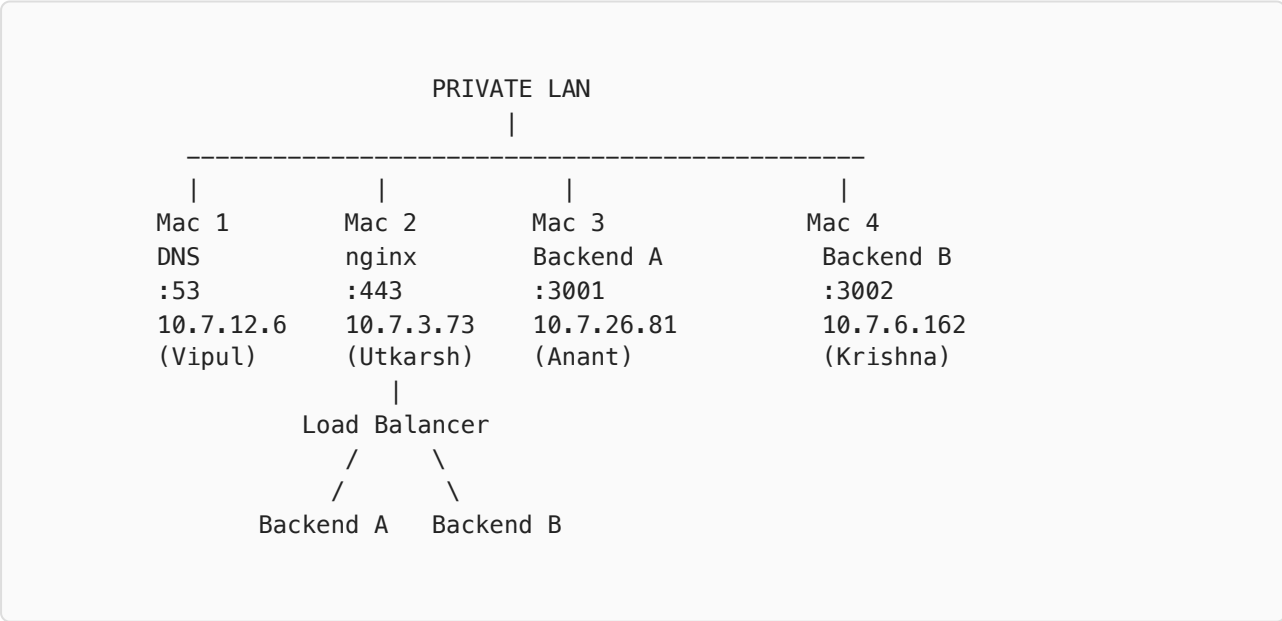
- Vipul Sharma — 2401010505 — Mac 1 (Private DNS)
- Utkarsh Jain — 2401020072 — Mac 2 (nginx Edge / TLS)
- Anant Jain — 2401010066 — Mac 3 (Backend A)
- Krishna Gehlot — 2401010236 — Mac 4 (Backend B / Client / Wireshark)

1. Introduction

This project implements a complete multi-machine networking infrastructure on a private LAN using four Apple Macs. The system demonstrates core computer networking concepts including private DNS resolution, reverse proxying with round-robin load balancing, TLS termination with a locally-signed certificate authority, HTTP caching mechanisms, and network packet capture analysis using Wireshark.

The architecture represents a simplified production web infrastructure where client requests traverse multiple network layers — DNS, TCP, TLS, and HTTP — before reaching the application backend, with all communication encrypted and load-balanced.

2. Architecture



Machine	Member	Role	IP Address	Port
Mac 1	Vipul Sharma	Private DNS (dnsmasq)	10.7.12.6	53
Mac 2	Utkarsh Jain	nginx Reverse Proxy / TLS	10.7.3.73	443
Mac 3	Anant Jain	Backend A (Node.js + Express)	10.7.26.81	3001
Mac 4	Krishna Gehlot	Backend B + Client + Wireshark	10.7.6.162	3002

2.1 Domain Configuration

Domain	Resolves To	Target
app.team1.test	10.7.3.73	nginx (Mac 2)
api.team1.test	10.7.3.73	nginx (Mac 2)

3. Request Flow

When a client makes a request to `https://app.team1.test/api/status`, the following sequence occurs:

- DNS Resolution** — The client queries Mac 1 (10.7.12.6:53). dnsmasq returns `10.7.3.73` for `app.team1.test`.
- TCP 3-Way Handshake** — The client initiates SYN → SYN-ACK → ACK with nginx on port 443.
- TLS Handshake** — Client Hello, Server Hello, certificate exchange. The server presents `app.crt` signed by Team1 Local CA. The client verifies the chain against the trusted `ca.crt`.
- HTTP Request (encrypted)** — `GET /api/status` is sent inside the TLS tunnel.
- nginx Proxy** — nginx selects a backend via round-robin and forwards the request to Backend A (10.7.26.81:3001) or Backend B (10.7.6.162:3002).
- Backend Response** — The backend returns JSON with an `X-Backend` header identifying itself.
- Response to Client** — nginx forwards the response back through the TLS connection.

4. Task A — IP Addressing & Connectivity

All four Macs are connected to the same private LAN. Connectivity was verified using ICMP ping between all machines.

```
ping -c 3 10.7.12.6    # Mac 1 — Vipul
ping -c 3 10.7.3.73    # Mac 2 — Utkarsh
ping -c 3 10.7.26.81   # Mac 3 — Anant
ping -c 3 10.7.6.162   # Mac 4 — Krishna
```

All pings returned successful replies, confirming Layer 3 (IP) connectivity.

5. Task B — Private DNS

Mac 1 (Vipul — 10.7.12.6) runs dnsmasq as a private DNS server. The configuration maps both custom domains to the nginx proxy:

```
listen-address=127.0.0.1,10.7.12.6
bind-interfaces
no-resolv
address=/app.team1.test/10.7.3.73
address=/api.team1.test/10.7.3.73
```

Client Macs set their DNS to 10.7.12.6 using `networksetup -setdnsservers Wi-Fi 10.7.12.6`.

Evidence — DNS Resolution

```
[vipulsharma@Vipuls-MacBook-Pro ~ % dig app.team1.test

; <<>> DiG 9.10.6 <<>> app.team1.test
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 49560
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
;; QUESTION SECTION:
;app.team1.test.                IN      A

;; ANSWER SECTION:
app.team1.test.                0       IN      A      10.7.3.73

;; Query time: 3 msec
;; SERVER: 10.7.12.6#53(10.7.12.6)
;; WHEN: Thu Oct 01 23:53:51 IST 2026
;; MSG SIZE rcvd: 59
```

Figure 1: `dig app.team1.test` — resolves to 10.7.3.73 via DNS server 10.7.12.6

6. Task C — Backend Servers

Both backends use the same Node.js + Express server (`server.js`). The backend name and port are passed as CLI arguments:

```
node server.js A 3001 # Mac 3 — Anant
node server.js B 3002 # Mac 4 — Krishna
```

The server listens on `0.0.0.0` for LAN accessibility and exposes three endpoints: `/`, `/api/status`, and `/api/cached`. Each response includes an `X-Backend` header.

Evidence — Backend A Direct Access

```
anantjain@Anants-MacBook-Pro ~ % curl -i http://10.7.26.81:3001/api/status
HTTP/1.1 200 OK
X-Backend: A
Cache-Control: no-cache
Content-Type: application/json; charset=utf-8
Content-Length: 29
ETag: W/"1d-/mMMZex8nvIDi44lSUobi0vFg/M"
Date: Thu, 01 Oct 2026 19:11:01 GMT
Connection: keep-alive
Keep-Alive: timeout=5

[{"backend":"A","status":"ok"}]
```

Figure 2: Direct `curl -i http://10.7.26.81:3001/api/status` from Anant's Mac — X-Backend: A

7. Task D — Load Balancing

nginx on Mac 2 (Utkarsh — 10.7.3.73) acts as a reverse proxy with round-robin load balancing:

```
upstream backends {
    server 10.7.26.81:3001 max_fails=1 fail_timeout=10s;
    server 10.7.6.162:3002 max_fails=1 fail_timeout=10s;
}
```

Requests alternate between Backend A and Backend B. The `max_fails=1 fail_timeout=10s` configuration enables automatic failover when a backend is unreachable.

Evidence — Load Balancing

```
→ ~ for i in 1 2 3 4 5 6; do
  /usr/bin/curl -s -i https://app.team1.test/api/status | grep -i "X-Backend"
done
X-Backend: B
X-Backend: A
X-Backend: B
X-Backend: A
X-Backend: B
X-Backend: A
→ ~
```

Figure 3: Six consecutive requests showing alternating X-Backend: B, A, B, A, B, A

```
[→ cn-backend curl -i http://10.7.26.81:3001/api/status
HTTP/1.1 200 OK
X-Backend: A
Cache-Control: no-cache
Content-Type: application/json; charset=utf-8
Content-Length: 29
ETag: W/"1d-/mMMZex8nvIDi44lSUobi0vFg/M"
Date: Thu, 01 Oct 2026 17:54:12 GMT
Connection: keep-alive
Keep-Alive: timeout=5

[{"backend":"A","status":"ok"}]
→ cn-backend for i in 1 2 3 4 5 6; do
  /usr/bin/curl -s -i https://app.team1.test/api/status | grep -i "X-Backend"
done
X-Backend: B
X-Backend: A
X-Backend: B
X-Backend: A
X-Backend: B
X-Backend: A
```

Figure 4: Direct backend access followed by load balancing test

8. Task E — HTTPS / TLS

A local Certificate Authority (Team1 Local CA) was created using OpenSSL. The server certificate covers both domains via Subject Alternative Names (SAN):

```
DNS.1 = app.team1.test  
DNS.2 = api.team1.test
```

The CA certificate (`ca.crt`) was installed on all client Macs using:

```
sudo security add-trusted-cert -d -r trustRoot \  
-k /Library/Keychains/System.keychain ca.crt
```

HTTPS verification was performed **without** the `-k` flag to prove proper certificate trust:

```
/usr/bin/curl -v https://app.team1.test/api/status
```

Evidence — HTTPS / TLS Handshake

```
[vipulsharma@Vipuls-MacBook-Pro ~ % curl -v https://app.team1.test/api/status
* Host app.team1.test:443 was resolved.
* IPv6: (none)
* IPv4: 10.7.3.73
* Trying 10.7.3.73:443...
* Connected to app.team1.test (10.7.3.73) port 443
* ALPN: curl offers h2,http/1.1
* (304) (OUT), TLS handshake, Client hello (1):
* CAfile: /etc/ssl/cert.pem
* CApath: none
* (304) (IN), TLS handshake, Server hello (2):
* (304) (IN), TLS handshake, Unknown (8):
* (304) (IN), TLS handshake, Certificate (11):
* (304) (IN), TLS handshake, CERT verify (15):
* (304) (IN), TLS handshake, Finished (20):
* (304) (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / AEAD-CHACHA20-POLY1305-SHA256 / [blank] / UNDEF
* ALPN: server accepted http/1.1
* Server certificate:
* subject: C=IN; O=Team1; CN=app.team1.test
* start date: Sep 30 10:55:16 2026 GMT
* expire date: Sep 30 10:55:16 2027 GMT
* subjectAltName: host "app.team1.test" matched cert's "app.team1.test"
* issuer: C=IN; O=Team1; CN=Team1 Local CA
* SSL certificate verify ok.
* using HTTP/1.x
> GET /api/status HTTP/1.1
> Host: app.team1.test
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: nginx/1.31.6
< Date: Wed, 30 Sep 2026 11:06:44 GMT
< Content-Type: application/json; charset=utf-8
< Content-Length: 29
< Connection: keep-alive
< X-Backend: A
< Cache-Control: no-cache
< ETag: W/"1d-/mMMZex8nvIDi441SUobi0vFg/M"
<
* Connection #0 to host app.team1.test left intact
{"backend":"A","status":"ok"}%
```

Figure 5: Full verbose curl showing TLS handshake, *SSL certificate verify ok*, certificate details (CN=app.team1.test, Issuer=Team1 Local CA), and 200 OK response

Evidence — nginx Config + Certificate

```

nginx: the configuration file /opt/homebrew/etc/nginx/nginx.conf syntax is ok
nginx: configuration file /opt/homebrew/etc/nginx/nginx.conf test is successful
[(base) utkarshjain@Utkarshs-MacBook-Pro-3 ~ % cat /opt/homebrew/etc/nginx/nginx.conf
events {}
http {
    upstream backends {
        server 10.7.26.81:3001 max_fails=1 fail_timeout=10s;
        server 10.7.6.162:3002 max_fails=1 fail_timeout=10s;
    }
    server {
        listen 80;
        server_name app.team1.test api.team1.test;
        location / {
            proxy_pass http://backends;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
    server {
        listen 443 ssl;
        server_name app.team1.test api.team1.test;

        ssl_certificate /Users/utkarshjain/team1-tls/app.crt;
        ssl_certificate_key /Users/utkarshjain/team1-tls/app.key;

        location / {
            proxy_pass http://backends;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
}
[(base) utkarshjain@Utkarshs-MacBook-Pro-3 ~ % openssl x509 -in ~/team1-tls/app.crt -noout -subject -issuer -dates
subject=C = IN, O = Team1, CN = app.team1.test
issuer=C = IN, O = Team1, CN = Team1 Local CA
notBefore=Sep 30 10:55:16 2026 GMT
notAfter=Sep 30 10:55:16 2027 GMT
(base) utkarshjain@Utkarshs-MacBook-Pro-3 ~ %
```

Figure 6: nginx configuration on Utkarsh's Mac with syntax test OK, and `openssl x509` showing certificate details

9. Task F — Caching

The `/api/cached` endpoint demonstrates HTTP caching:

- `Cache-Control: max-age=60` — instructs clients to cache for 60 seconds
- `ETag` — auto-generated by Express as a fingerprint of the response body

When a client sends `If-None-Match` with the previously received ETag, the server responds with **304 Not Modified** instead of retransmitting the data, saving bandwidth.

Evidence — Cache Response

```
[vipulsharma@Vipuls-MacBook-Pro ~ % curl -i https://app.team1.test/api/cached
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Thu, 01 Oct 2026 03:47:31 GMT
Content-Type: application/json; charset=utf-8
Content-Length: 25
Connection: keep-alive
X-Backend: B
Cache-Control: max-age=60
ETag: W/"19-3Xq2iFjXnMbnAKEagMsqMcjEGaE"
```

Figure 7: `curl -i https://app.team1.test/api/cached` — 200 OK with Cache-Control: max-age=60, ETag, X-Backend: B

10. Task G — Wireshark Packet Capture

Packet capture was performed on Mac 4 (Krishna — 10.7.6.162) using Wireshark on interface `en0`.

To capture a TLS 1.2 session for clearer analysis:

```
/usr/bin/curl -v --tlsv1.2 --tls-max 1.2 https://app.team1.test/api/status
```

Display Filters Used

Filter	Purpose
<code>dns</code>	DNS query and response
<code>tcp.flags.syn==1 && tcp.port==443</code>	TCP SYN packets for HTTPS connections
<code>tls.handshake</code>	TLS handshake messages
<code>tls.record.content_type == 23</code>	Encrypted application data
<code>tls && ip.addr == 10.7.3.73 && tcp.port == 443</code>	Complete TLS session to nginx

Evidence — Wireshark TLS Session

Figure 8 shows a Wireshark capture of a TLS session. The filter applied is `tls && ip.addr == 10.7.3.73 && tcp.port == 443`. The packet list displays the following packets:

No.	Time	Source	Destination	Protocol	Length	Info
13523	285.827812	10.7.6.162	10.7.3.73	TLSv1...	390	Client Hello (SNI=app.team1.test)
13526	285.873750	10.7.3.73	10.7.6.162	TLSv1...	1514	Server Hello, Change Cipher Spec, Application Data, Application Data
13527	285.873756	10.7.3.73	10.7.6.162	TLSv1...	362	Application Data, Application Data
13531	286.023407	10.7.6.162	10.7.3.73	TLSv1...	90	Application Data
21152	487.053095	10.7.6.162	10.7.3.73	TLSv1...	390	Client Hello (SNI=app.team1.test)
21155	487.086410	10.7.3.73	10.7.6.162	TLSv1...	1514	Server Hello, Change Cipher Spec, Application Data, Application Data
21156	487.086430	10.7.3.73	10.7.6.162	TLSv1...	362	Application Data, Application Data
21159	487.141269	10.7.6.162	10.7.3.73	TLSv1...	90	Application Data

The packet details for frame 13523 (Client Hello) are expanded, showing the Ethernet II header, Internet Protocol Version 4 header, and the TCP segment. The TCP segment is a Client Hello with the following details:

- Version: 4
- Header Length: 20 bytes (5)
- Differentiated Services Field: 0x02 (DSCP: CS0, ECN: ECT(0))
- Total Length: 376
- Identification: 0x0000 (0)
- Flags: 0x02, Don't fragment
- Fragment Offset: 0
- Time to Live: 64
- Protocol: TCP (6)
- Header Checksum: 0xb86 [validation disabled]
- Header checksum status: Unverified
- Source Address: 10.7.6.162

Figure 8: Wireshark filter `tls && ip.addr == 10.7.3.73 && tcp.port == 443` — Client Hello (SNI=app.team1.test), Server Hello, Application Data

Evidence — TCP + TLS Lifecycle

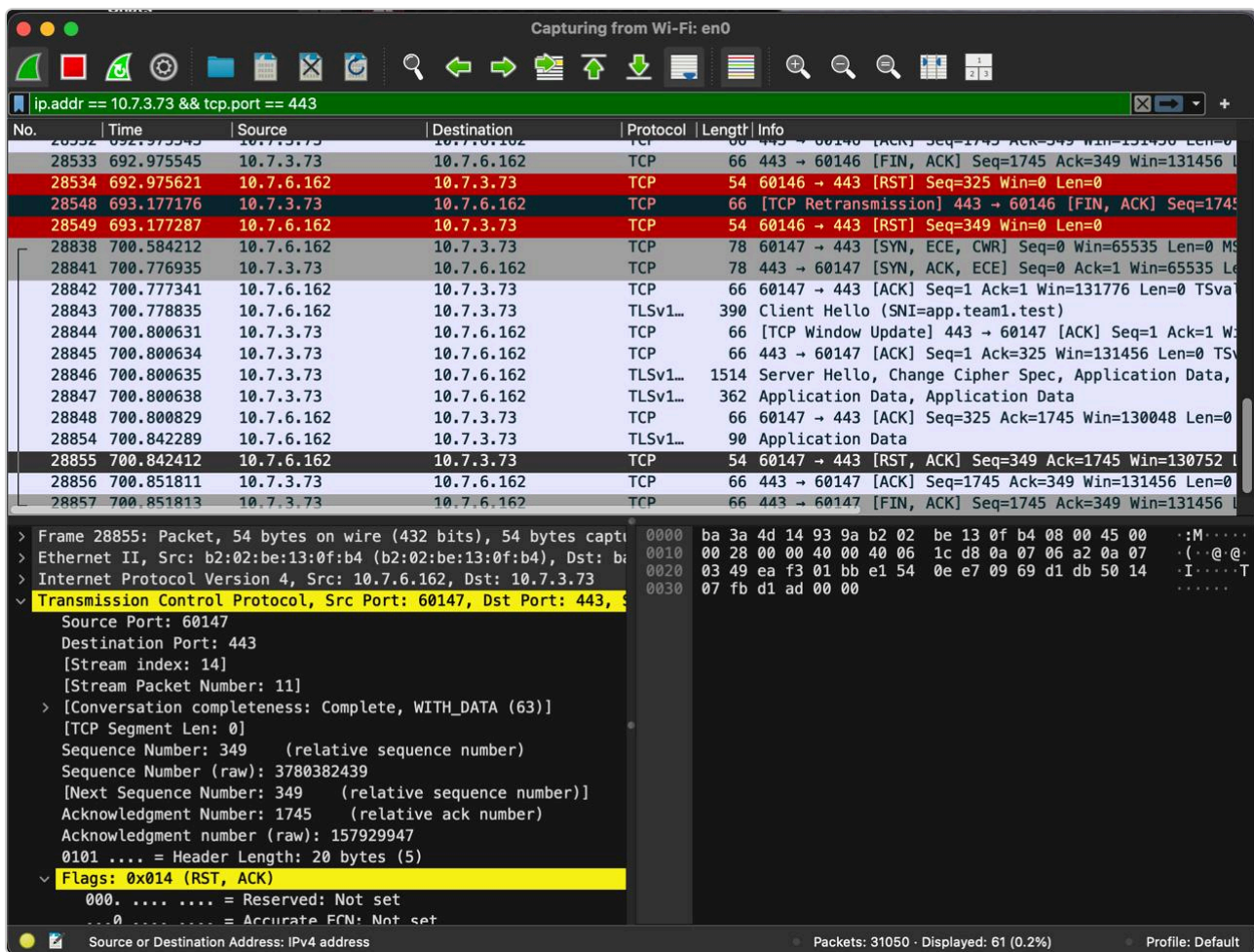


Figure 9: Complete TCP + TLS connection lifecycle showing SYN, Client Hello, Server Hello, Application Data, RST/ACK, FIN/ACK

Evidence — TCP SYN

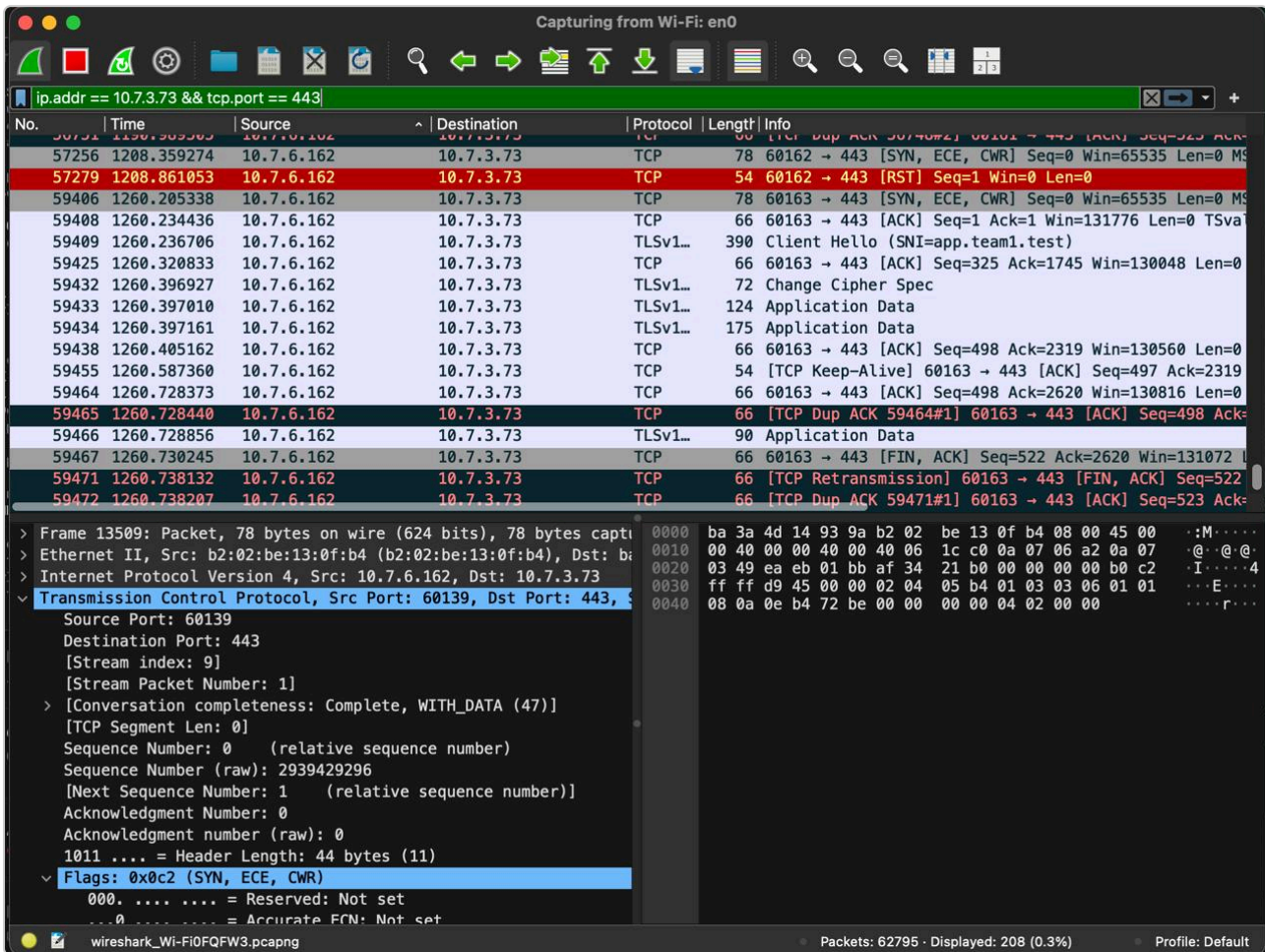


Figure 10: TCP SYN (SYN, ECE, CWR) establishing connection to port 443, followed by TLS handshake with Change Cipher Spec

Evidence — TLS Overview

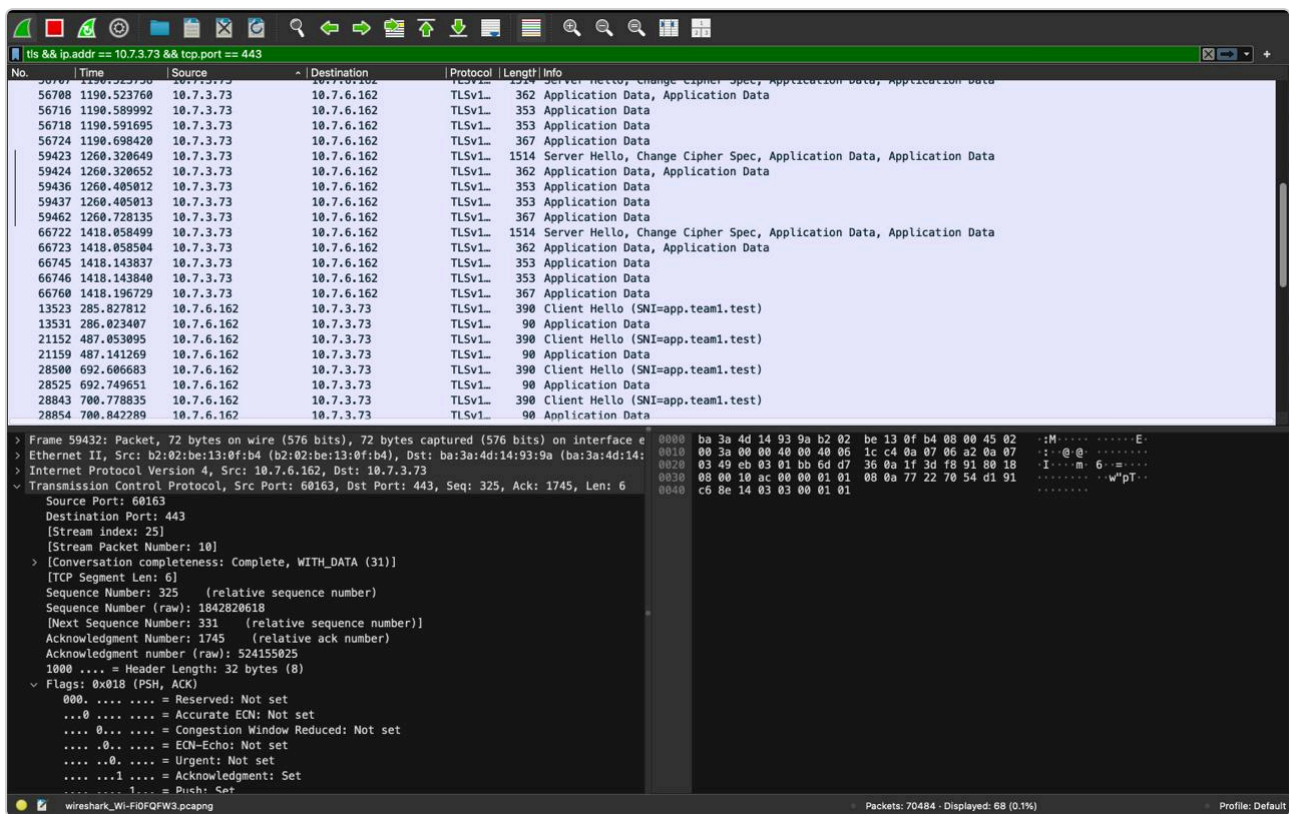


Figure 11: Overview of multiple TLS sessions — Client Hello (SNI=app.team1.test), Server Hello, and encrypted Application Data

11. Failure Demonstrations

Five failure scenarios were demonstrated to prove understanding of the network stack:

#	Failure Scenario	Broken Layer	Expected Result
1	Wrong DNS server (8.8.8.8 instead of 10.7.12.6)	DNS	NXDOMAIN — <code>app.team1.test</code> not resolved
2	DNS points to wrong IP (10.7.26.81 instead of 10.7.3.73)	DNS → TCP	Connection refused — no nginx on that IP
3	One backend stopped	Application	nginx failover — all responses from healthy backend
4	Both backends stopped	Application	502 Bad Gateway — no upstream available
5	Wrong HTTPS port (8444 instead of 443)	TCP	Connection refused — nothing listening

Failure 1 — Wrong DNS Server

When the client DNS is set to `8.8.8.8` (Google), `app.team1.test` fails to resolve because it is a private domain. Direct IP access (ping) still works, proving the failure is isolated to the DNS layer.

Failure 2 — Wrong DNS Record

When `app.team1.test` resolves to `10.7.26.81` (Mac 3) instead of `10.7.3.73` (Mac 2), DNS resolution succeeds but TCP connection to port 443 fails because Mac 3 does not run nginx.

Failure 3 — One Backend Down

When Backend A is stopped, nginx detects the failure (via `max_fails=1`) and routes all traffic to Backend B. All responses show `X-Backend: B`. When Backend A restarts, normal round-robin resumes after `fail_timeout=10s`.

Failure 4 — Both Backends Down

When both backends are stopped, nginx returns `502 Bad Gateway`. DNS and TLS still work correctly — the failure is isolated between nginx and the upstream servers.

Failure 5 — Wrong HTTPS Port

`curl https://app.team1.test:8444/` results in connection refused. Port 8444 is not open on Mac 2. ICMP ping to the same host still succeeds, proving the issue is at the transport layer.

12. Conclusion

This project successfully demonstrates a multi-layer network architecture on a private LAN. The system implements DNS resolution, TCP/TLS connections, reverse proxying with load balancing, HTTP caching, and provides Wireshark evidence of all network protocols in action. The five failure demonstrations prove isolation and understanding of each network layer.

CN Project Phase 1 — Team Jarvis 07

Vipul Sharma (2401010505) · Utkarsh Jain (2401020072) · Anant Jain (2401010066) · Krishna Gehlot (2401010236)